



分支结构综合

嵌套 · 陷阱 · 实战技巧



为什么需要嵌套分支？



有时一个条件的判断依赖于另一个条件的结果：

- 先判断是否成年，再判断是否允许进入某些场所
- 先判断是否及格，再判断是否优秀
- 先判断是否闰年，再判断二月天数

嵌套分支 让程序能够处理更复杂的逻辑关系。

分支嵌套的基本格式



```
if (条件1) {  
    // 外层条件为真  
    if (条件2) {  
        // 内层条件也为真  
    } else {  
        // 内层条件为假  
    }  
} else {  
    // 外层条件为假  
    if (条件3) {  
        // 内层条件为真  
    }  
}
```

关键点

- 内层 `if` 只在外层条件成立时才执行
- 缩进要清晰，体现层次关系
- 每一层都有自己独立的 `else` 配对



嵌套示例：判断成绩等级和是否及格



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int score;
    cin >> score;

    if (score >= 0 && score <= 100) {
        // 外层：判断是否合法
        if (score >= 60) {
            cout << "及格" << endl;
            if (score >= 90) {
                cout << "等级: A" << endl;
            } else if (score >= 80) {
                cout << "等级: B" << endl;
            } else {
                cout << "等级: C" << endl;
            }
        } else {
            cout << "不及格" << endl;
        }
    } else {
        cout << "非法分数" << endl;
    }
    return 0;
}
```

例题1：判断闰年（嵌套条件）



题目描述

输入一个年份，判断它是否为闰年。闰年的规则：

- 能被4整除但不能被100整除，或者能被400整除。

输入：一个整数 `year` ($1900 \leq year \leq 2100$)

输出：`"闰年"` 或 `"平年"`

样例

输入：`2000` → 输出：`闰年`

输入：`1900` → 输出：`平年`

输入：`2024` → 输出：`闰年`

例题1 解析



考查点：分支嵌套、逻辑运算符组合、`&&` 和 `||` 的优先级。

思路：

- 先判断是否能被4整除，若不能则直接平年。
 - 若不能被100整除，则是闰年。
- 若能，再进一步判断是否能被100整除：
 - 若不能被100整除，则是平年。
 - 若能被100整除，则必须能被400整除才是闰年。

陷阱提醒：

- 勿将 `==` 写成 `=`（赋值）。
- 逻辑表达式建议加括号明确优先级：`(year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)`。



例题1 参考代码 (嵌套 if)



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int year;
    cin >> year;
    if (year % 4 == 0) {
        if (year % 100 != 0) {
            cout << "闰年";
        } else {
            if (year % 400 == 0)
                cout << "闰年";
            else
                cout << "平年";
        }
    } else {
        cout << "平年";
    }
    return 0;
}
```

更简洁的写法（逻辑表达式）：



```
if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
    cout << "闰年";
else
    cout << "平年";
```

例题2：三个数排序（嵌套比较）



题目描述

输入三个整数，按从小到大（升序）输出。

输入：三个整数 `a, b, c`

输出：升序排列的三个数，空格隔开

样例

输入： `3 1 2` → 输出： `1 2 3`

输入： `5 5 4` → 输出： `4 5 5`

例题2 解析



考查点：嵌套 `if` 进行两两比较，交换变量。

思路：

- 先保证 `a ≤ b`，如果 `a > b` 则交换。
- 再保证 `a ≤ c`，如果 `a > c` 则交换（此时 `a` 已是最小值）。
- 最后保证 `b ≤ c`，如果 `b > c` 则交换。

注意：交换时需要使用临时变量，或者用 `swap()` 函数（C++ STL）。

常见错误：忘记交换后的连锁影响，或者比较条件写错。

例题2 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    // 使 a 最小
    if (a > b) swap(a, b);
    if (a > c) swap(a, c);
    // 此时 a 已最小, 再比较 b 和 c
    if (b > c) swap(b, c);
    cout << a << " " << b << " " << c;
    return 0;
}
```

注: `swap()` 在 `<algorithm>` 中, `bits/stdc++.h` 已包含。

例题3：分段函数（嵌套 if）



题目描述

计算分段函数的值：

$$x + 1, x < 0$$

$$y = 2x, 0 \leq x < 5$$

$$3x - 5, x \geq 5$$

text

输入一个实数 x ，输出对应的 y 值（保留两位小数）。

输入：一个实数 x

输出： y 的值，保留两位小数

样例

输入：-1 → 输出：0.00

输入：3 → 输出：6.00

输入：5 → 输出：10.00



例题3 解析



考查点：区间判断顺序，嵌套 `if-else`。

思路：

- 使用 `if-else if-else` 结构，注意条件顺序。
- 由于区间是连续的，顺序很重要：先判断 `x < 0`，再 `x < 5`（隐含 $x \geq 0$ ），最后 `else` 表示 $x \geq 5$ 。

陷阱：

- 浮点数比较应避免直接 `==`，但本题用 `<` 和 `>=` 无问题。
- 输出格式：`cout << fixed << setprecision(2) << y;`

例题3 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    double x, y;
    cin >> x;
    if (x < 0) {
        y = x + 1;
    } else if (x < 5) {    // 隐含 x >= 0
        y = 2 * x;
    } else {              // x >= 5
        y = 3 * x - 5;
    }
    cout << fixed << setprecision(2) << y;
    return 0;
}
```

例题4：判断三角形类型（嵌套）



题目描述

输入三个整数（代表三条边长），判断能否构成三角形，若能则进一步判断是等边、等腰还是普通三角形。

输入：三个整数 `a, b, c`

输出：`"等边三角形"` / `"等腰三角形"` / `"普通三角形"` / `"不能构成三角形"`

样例

输入：`3 3 3` → `等边三角形`

输入：`3 4 5` → `普通三角形`

输入：`1 1 3` → `不能构成三角形`

例题4 解析



考查点：嵌套 `if` 与逻辑运算符的组合。

思路：

- 先判断是否能构成三角形（任意两边之和大于第三边）。
 - 三边相等 $\rightarrow$ 等边
 - 有两边相等 $\rightarrow$ 等腰（注意等边也是等腰，但先判断等边可避免歧义）
- 若能，再判断三边关系：
 - 否则普通。

常见陷阱：

- 忘记判断三角形合法性，直接分类。
- 使用 `a == b == c` 错误（应写 `a == b && b == c`）。

例题4 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    if (a + b > c && a + c > b && b + c > a) {
        if (a == b && b == c) {
            cout << "等边三角形";
        } else if (a == b || a == c || b == c) {
            cout << "等腰三角形";
        } else {
            cout << "普通三角形";
        }
    } else {
        cout << "不能构成三角形";
    }
    return 0;
}
```

例题5：计算器（带除零和非法运算符）



题目描述

输入两个整数和一个运算符（`+` `-` `*` `/` `%`），输出运算结果。注意除法和取余的除数为0时输出

`"Error"`，运算符非法时输出 `"Invalid operator"`。

输入： `a op b` （如 `10 + 5`）

输出： 运算结果或错误信息

样例

输入： `10 / 3` → `3`

输入： `10 % 0` → `Error`

输入： `10 # 3` → `Invalid operator`

例题5 解析



考查点： `switch` 与嵌套 `if` 结合，注意 `break` 和 `default`。

思路：

- 使用 `switch(op)` 分别处理各运算符。
- 对于 `/` 和 `%`，内部嵌套 `if` 判断除数是否为0。
- `default` 处理非法运算符。

陷阱：

- `switch` 中漏写 `break` 导致穿透。
- 注意 `%` 要求整数，已满足。
- 赋值与相等混淆（如 `if (b = 0)`）。



例题5 参考代码 (switch + 嵌套)



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    char op;
    cin >> a >> op >> b;
    switch (op) {
        case '+': cout << a + b; break;
        case '-': cout << a - b; break;
        case '*': cout << a * b; break;
        case '/':
            if (b == 0) cout << "Error";
            else cout << a / b;
            break;
        case '%':
            if (b == 0) cout << "Error";
            else cout << a % b;
            break;
        default: cout << "Invalid operator";
    }
    return 0;
}
```

例题6：字符分类（嵌套判断）



题目描述

输入一个字符，判断它是大写字母、小写字母、数字还是其他字符。

输入：一个字符 `c`

输出：`"upper"`、`"lower"`、`"digit"` 或 `"other"`

样例

输入：`G` → `upper`

输入：`5` → `digit`

输入：`@` → `other`

例题6 解析



考查点：字符比较，`if-else if` 结构。

思路：

- 利用 ASCII 码：大写 `'A' ~ 'Z'`，小写 `'a' ~ 'z'`，数字 `'0' ~ '9'`。
- 也可以用 `c >= 'A' && c <= 'Z'` 等。
- 注意 `char` 输入会读取空白符，可忽略。

陷阱：

- 误将字符与数字比较（如 `c >= 65` 也可，但可读性差）。
- 忘记 `else` 导致多输出。



例题6 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    char c;
    cin >> c;
    if (c >= 'A' && c <= 'Z') {
        cout << "upper";
    } else if (c >= 'a' && c <= 'z') {
        cout << "lower";
    } else if (c >= '0' && c <= '9') {
        cout << "digit";
    } else {
        cout << "other";
    }
    return 0;
}
```

例题7：判断正负奇偶（嵌套）



题目描述

输入一个整数，判断它的正负性和奇偶性，输出四种情况之一："正奇数"、"正偶数"、"负奇数"、"负偶数"（零视为正偶数）。

输入：一个整数 `n`

输出：对应类别

样例

输入：5 → 正奇数

输入：-4 → 负偶数

输入：0 → 正偶数

例题7 解析



考查点：嵌套 `if` 处理多维度分类。

思路：

- 先判断正负（`n > 0`，`n < 0`，`n == 0`）。
- 再在正负分支内判断奇偶（`n % 2 == 0`）。
- 注意零的奇偶性一般视为偶数。

常见错误：

- 在判断奇偶时用 `n % 2 == 1` 判断奇数，但对于负数可能得到 -1，应使用 `n % 2 != 0`。
- 忽略零的情况导致逻辑漏洞。



例题7 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    if (n > 0) {
        if (n % 2 == 0)
            cout << "正偶数";
        else
            cout << "正奇数";
    } else if (n < 0) {
        if (n % 2 == 0)    // 负偶数
            cout << "负偶数";
        else
            cout << "负奇数";
    } else {
        cout << "正偶数"; // 0 视为正偶数
    }
    return 0;
}
```

例题8：月份天数（嵌套处理闰年）



题目描述

输入年份和月份，输出该月的天数（考虑闰年）。

输入：两个整数 `year month`

输出：该月的天数，若月份非法输出 `"Invalid month"`

样例

输入： `2024 2` → `29`

输入： `2023 2` → `28`

输入： `2023 13` → `Invalid month`

例题8 解析



考查点： `switch` 中嵌套 `if` 判断闰年。

思路：

- 用 `switch(month)` 对月份分类：大月31天，小月30天，2月特殊。
- 2月内嵌套 `if` 判断闰年。
- `default` 处理非法月份。

陷阱：

- 注意 `case` 的堆叠写法（穿透利用）。
- 闰年判断公式容易写错（`%100` 和 `%400` 的关系）。



例题8 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int year, month;
    cin >> year >> month;
    int days;
    switch (month) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            days = 31; break;
        case 4: case 6: case 9: case 11:
            days = 30; break;
        case 2:
            if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0)
                days = 29;
            else
                days = 28;
            break;
        default:
            days = -1; // 标记非法
    }
    if (days == -1)
        cout << "Invalid month";
    else
        cout << days;
    return 0;
}
```

例题9：ATM 取款模拟（嵌套判断）



题目描述

输入账户余额（整数）和取款金额，判断是否能取款。

- 若余额不足，输出 "余额不足"
- 若取款金额为负数或零，输出 "金额无效"
- 否则输出取款后的余额。

输入：两个整数 `balance withdraw`

输出：取款后余额或错误信息

样例

输入： `1000 200` → `800`

输入： `500 600` → `余额不足`

输入： `500 -10` → `金额无效`

例题9 解析



考查点：多级嵌套 `if` 处理业务逻辑。

思路：

- 先判断取款金额是否合法（`> 0`）。
- 再判断余额是否足够（`withdraw <= balance`）。
- 最后计算余额。

陷阱：

- 判断顺序：应先判断金额合法性，再判断余额，避免非法金额导致余额比较出错。
- 注意 `=` 与 `==` 的误用。



例题9 参考代码



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int balance, withdraw;
    cin >> balance >> withdraw;
    if (withdraw <= 0) {
        cout << "金额无效";
    } else {
        if (withdraw > balance) {
            cout << "余额不足";
        } else {
            balance -= withdraw;
            cout << balance;
        }
    }
    return 0;
}
```

例题10：最大最小值（嵌套比较）



题目描述

输入三个整数，输出其中的最大值和最小值。

输入：三个整数 `a, b, c`

输出："`最大值 x, 最小值 y`"

样例

输入：`3 7 5` → `最大值 7, 最小值 3`

输入：`-1 -5 0` → `最大值 0, 最小值 -5`

例题10 解析



考查点：嵌套 `if` 或逻辑运算符实现极值。

思路：

- 先假设 `a` 为最大值和最小值，然后依次与 `b`、`c` 比较并更新。
- 使用嵌套 `if` 或链式比较。

陷阱：

- 同时更新最大值和最小值时注意不要用未初始化的变量。
- 也可以用 `max` 和 `min` 函数，但本题要求体现分支结构。



例题10 参考代码 (嵌套 if)



```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, c;
    cin >> a >> b >> c;
    int max_val = a, min_val = a;
    if (b > max_val) max_val = b;
    else if (b < min_val) min_val = b;
    // 注意: 上面的 else if 只会执行一个, 可能漏判, 应分开写
    // 正确写法
```

或者用嵌套比较实现:

```
int max_val, min_val;
if (a > b) {
    max_val = a;
    min_val = b;
} else {
    max_val = b;
```



总结

- **分支嵌套**：在分支内部再写分支，解决复杂判断。
 - `=` 赋值 与 `==` 相等 混淆
 - 悬空 `else` (`else` 与最近的 `if` 配对)
- **常见陷阱**：
 - `switch` 漏写 `break`
- **练习**：多写代码，注意逻辑完整性。

谢谢！